Instructor Handout 4: Registers, Instructions, and the Instruction Cycle

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

August 8, 2026

Instructor Copy — Not for Student Distribution

4.1 Before You Teach This Lecture

- **Mano, *Computer System Architecture*, 3rd ed.** — Sec. 8-1 "Introduction," p.241 (CPU structure); Sec. 8-2 "General Register Organization," p.242 and Table 8-1, p.243 (register-selection fields `SELA`/`SELB`/`SELD`); Fig. 5-4, p.129 (registers and memory on the common bus system); Sec. 8-3 "Stack Organization," pp. 247–248 (definition, register vs. memory stack); Sec. 8-4 "Instruction Formats," p.255. All page-cited — the Mano index is built and verified (2026-08-06).
- **Patterson & Hennessy, *Computer Organization and Design*, ARM Edition** — Sec. 2.3 "Operands of the Computer Hardware," p.67 (register set, memory operands, constants); Sec. 2.5 "Representing Instructions in the Computer," p.82 (fixed-width fields, formats); Sec. 2.7 "Instructions for Making Decisions," p.93 (branches, the 4-byte instruction point); Sec. 2.8 "Supporting Procedures," p.100 (stack-frame convention, `SP`). Page-cited — index built 2026-08-07. P&H gives you a concrete RISC reference point (LEGv8) to contrast with the accumulator/stack abstractions.
- **Hamacher, *Computer Organization*, Ch. 7** — *not yet page-indexed.* Cite at chapter level; the source for the instruction-cycle skeleton (fetch/decode/execute) and the four operand types.
- **Reread Week 3's pivot** (numbers + ALU are the *data* layer) — the opening two minutes set up that this week supplies the *machine* that animates those bits. Also note the running thread you will carry all lecture: $X = (A + B) \times (C + D)$ compiled in every format.

4.2 Register Organization

Open by asking the class where `X`, `A`, `B`, `C`, `D` live while a program runs. The point to extract before showing anything: values must *move* from storage to the ALU's inputs and back, and the three candidate homes differ wildly in speed — main memory is big but slow (a read takes many CPU cycles), registers are tiny but fastest (one cycle), and the compiler decides what earns a register.

Definition (Program-Visible Register). *A register an instruction can name as an operand or a result location (e.g. `R0..Rn-1`, `AC`, `SP`) — as opposed to the internal registers (`PC`, `IR`, `MAR`, `MDR`) that only the control unit reads and writes.*

Then draw the CPU as three parts and one relationship (Figure 4.1).

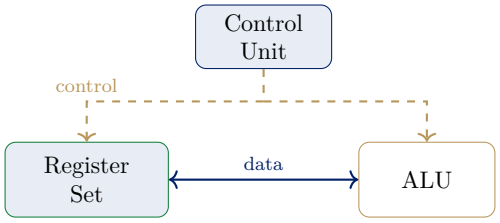

Figure 4.1: The CPU's major components — cf. Mano, Sec. 8-1, p.241: register set (holds values), ALU (computes), control unit (decides what happens when — built in Weeks 6–7).

How to present it: draw the two boxes that actually move data first (`Register Set` and `ALU` side by side), connect them with the horizontal data path, then draw the `Control Unit` on top as the third, still-unexplained box. Only after the class has puzzled over "what is that top box for?" do you reveal it as the control unit and say plainly: Weeks 6–7 build it. That ordering makes the two-week tease land instead of being announced.

> **Teaching note:** A common conflation: students think the `Control Unit` computes something. Kill this now — its job is to *decide* and *gate*, not to do arithmetic. The ALU does arithmetic; the control unit just tells it what to do. The cleanest one-line question: "if the control unit breaks, can the ALU still add?" (Yes — it just never gets told to.)

List the program-visible registers (Mano, Sec. 8-2, p.242) *as a class answer*, then the internal ones. Ask "which registers could a program name in an instruction, if it wanted?" before revealing the split. `R0..Rn-1` (general-purpose, any register can hold an address or a value), `AC` (implicit accumulator for one-address ISAs), `SP` (stack pointer — a whole section coming). Internal: `PC`, `IR`, `MAR`, `MDR` — the program never names them; the control unit drives them. P&H's LEGv8 gives the concrete RISC analogue: architectural `X0`–`X30` plus `SP` is what a compiler names (P&H, Sec. 2.3, p.67).

4.2.1 The Common Bus System and Register Selection

The registers do not talk to each other directly — one shared path, the **common bus**, carries values among registers, ALU, and memory (Figure 4.2), and a **select** unit (decoder) chooses, each cycle, *which* register drives the bus.

How to present it: draw the single horizontal bus first, labeled *common bus*, then the four registers above it one at a time, each with its vertical connection, then the ALU and the `MAR`/`MDR` memory interface on the right, then the `select logic` box below — saving its label for last, so the class guesses what it does. The vertical "tap" into the ALU should be centered under the ALU's column so it reads cleanly.

> **Teaching note:** Students routinely ask why every register needs its own wire to the bus rather than "just connecting R1 to R2 directly." The answer is exactly the bus's reason to exist: n registers with all pairwise wires is $n(n-1)/2$ wires, but one bus is n taps. Say the count out loud — it is the same "shared medium" argument as a class room's single whiteboard.

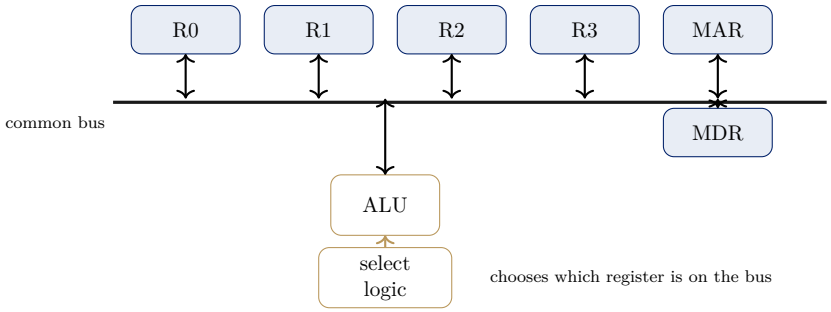

Figure 4.2: The common bus system — cf. Mano, Fig. 5-4, p.129. One shared path carries values among registers, ALU, and memory; the select unit (decoder) chooses which register drives the bus each cycle.

Now the selection mechanism (Mano, Table 8-1, p.243): `SELA` names which register's contents go to the ALU's *first* input; `SELB` names which register's contents go to the ALU's *second* input; `SELD` names *which* register is loaded with the result. The idea: register selection is *encoded* in a few select bits and decoded into one-hot "drive bus" / "load now" signals — that is the whole trick behind moving values between registers without hard-wiring every pair.

Example 4.2.1 (`R1 ← [R2]` over the common bus)**.** *Copy the* contents *of `R2` into `R1` (contents of register X are written* $[X]$*):*

1. *`SELA` encodes "`R2`" — R2's output enable goes high.*
2. *`R2`'s contents are driven onto the common bus.*
3. *`SELD` encodes "`R1`" — R1's load-enable goes high.*
4. *On the clock edge, `R1` captures whatever the bus holds:* $R1 \leftarrow [R2]$.

No direct `R2`→`R1` wire exists — every transfer uses the bus plus two select signals.

A question worth asking live, after the trace: "what would change if we wanted `R1 ← R3` instead?" (Nothing structural — only the encoded values of `SELA` and `SELD` change.) This is the payoff question that shows the bus + select mechanism is generic.

> **Teaching note:** The single most likely point of confusion is conflating `SELA`/`SELB` (which *sources* feed the ALU) with `SELD` (which register *receives* the result). Drill the asymmetry: two source-select fields, one destination-select field, because an ALU operation has two operands but one result.

Example 4.2.2 (A Second Instance: `R3 ← [R0] + [R2]`)**.** 1. *`SELA` = "`R0`" — R0's contents driven onto the bus into ALU input A.*

2. *`SELB` = "`R2`" — R2's contents driven onto the bus into ALU input B.*
3. *ALU set to add; `SELD` = "`R3`" — R3's load-enable goes high.*
4. *On the clock edge,* $R3 \leftarrow [R0] + [R2]$.

Identical machinery to the copy example — the only new element is the ALU's select line between steps 2 and 3.

Close the section with the Socratic check: why not put *everything* in registers? Registers are scarce (a few dozen at most), costly (silicon area + power for each), so they are a scratchpad for what is *being* worked on; values rest in memory (big, cheap, persistent) and are loaded in only while the ALU touches them. Ask the question live and let the class produce "scarcity" and "cost" before you confirm.

4.3 Stack Organization

Pose the postfix puzzle: how would a calculator evaluate `A B +` if it could only look at the *top* item of a scratch list — no registers, no random access? The rule: read a value, **push** it; read an operator, **pop** the last two values, compute, **push** the result. The *last* value pushed is the *first* taken back out — **last-in, first-out** (LIFO). That is a **stack**. Name the machines that use it: HP calculators, the JVM.

Definition (Stack). *An ordered storage structure where all insertions and deletions happen at* one end*, the* ***top****, addressed by a dedicated register* `SP` *(the* ***stack pointer****); behavior is LIFO (Mano, Sec. 8-3, p.247).*

PUSH: $SP \leftarrow [SP] - 1$, then $M[SP] \leftarrow$ operand — store at the new top (the stack grows *downward* toward lower addresses). **POP:** operand $\leftarrow M[SP]$, then $SP \leftarrow [SP] + 1$ — remove the current top.

> **Teaching note:** The downward growth (decrement on PUSH, increment on POP) is the #1 intuition inversion of the whole lecture — every student assumes a stack grows *up*. State the convention loudly and give the reason: the stack shares memory with the rest of the program, and growing downward from a high base keeps it from colliding with the low-addressed program/data region.

Example 4.3.1 (PUSH and POP micro-transfers)**.** *Stack contains* `[? , 5 , 7]` *with top at 7.* `PUSH 9` $\Rightarrow$ `[? , 5 , 7 , 9]`*;* `POP` $\Rightarrow$ *returns 9, stack back to* `[? , 5 , 7]`*.*

Then run the `A + B` stack trace (Table 1) and stop at the ADD row: this instruction names *no operands at all* — both come implicitly from the top of the stack. That is the **zero-address** format, met properly in the next section.

Step	Operation	Stack after (top right)	Effect
1	`PUSH A`	`A`	value A stored at top
2	`PUSH B`	`B A`	value B stored above A
3	`ADD`	`A+B`	pop B, pop A, push result
4	`POP X`	*empty*	store top into `X`

Table 1: `A + B` on a stack machine — the `ADD` names no operands; both come from the stack top.

Now the two hardware homes (Mano, Sec. 8-3, p.248): a **register stack** lives inside the CPU in a set of registers — very fast but only a handful of words (e.g. 64); a **memory stack** is a region of main memory with `SP` holding the address of the top — as large as memory, but every PUSH/POP is a memory access. Real machines choose memory stacks because function calls nest deeply (recursion); real ISAs keep `SP` in a register and store frames in memory (P&H, Sec. 2.8, p.100). Say the punchline: registers are the pointer, memory is the stack.

Example 4.3.2 (A Second Instance: `A − B` on a stack machine)**.** *`PUSH A; PUSH B; SUB; POP X` — the `SUB` again names no operands. Worth asking: which order does the ALU pop them in, so that $A - B$ (not $B - A$) is computed? (The* second-*popped value is the minuend — convention matters and is worth one live question.)*

Close with the registers-vs-stack check: a stack restricts access (only the top — no random access mid-computation), costs reordering when a buried value is needed (pop everything above it first), but shrinks instructions (operands implicit — hence the compact zero-address format). Compilers love the compactness; hardware pays with restricted access.

4.4 Instruction Formats

Pose the question that defines the section: the CPU must be told *what* to do and *where* the operands are. How much "where" information must a machine instruction physically carry? At minimum an **opcode**; beyond that, *addresses* — and each address costs bits. The trade-off is fundamental: more addresses per instruction ⇒ more work done per instruction but longer instructions; fewer addresses ⇒ compact but more instructions, and the hardware must know where the operands are implicitly.

Definition (Instruction Format). *The layout of an instruction word: an **opcode** field naming the operation, followed by zero or more **address/operand** fields naming where operands and results live (Mano, Sec. 8-4, p.255).*

Fields are fixed-width bits; the opcode width sets an upper bound on how many distinct instructions the ISA can name. P&H's LEGv8 shows the fixed-width idea concretely — instructions in a handful of formats (R/I/IS/B/CB/IM), each with fixed fields (P&H, Sec. 2.5, p.82).

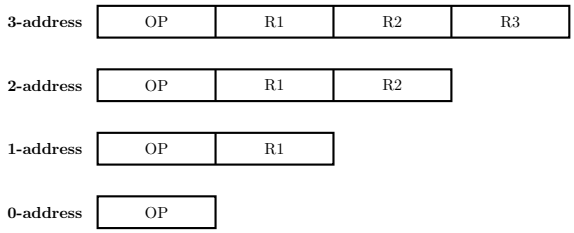

Figure 4.3: The four instruction-format families — cf. Mano, Sec. 8-4, p.255: as the operand count falls, instructions get shorter but the machine must find operands implicitly.

How to present it: draw the four rows *top to bottom* and read them as a compression story. Draw the 3-address row fully (OP + three fields), then ask the class to predict what the 2-address row drops, and so on down to 0-address (OP only). The visual of words shrinking while "implicit operand" grows is the entire section in one figure.

State each family's rule clearly:

- **3-address:** `ADD R1, R2, R3` ⇒ $R1 \leftarrow [R2] + [R3]$ — all explicit; the most flexible, longest form; register ISAs (RISC).
- **2-address:** `ADD R1, R2` ⇒ $R1 \leftarrow [R1] + [R2]$ — the first field doubles as one operand *and* the result; **destructive** (old `R1` lost); older CISC.
- **1-address:** `ADD A` ⇒ $AC \leftarrow [AC] + [A]$ — one operand is always the accumulator `AC`, named by no field; simple CPUs.

- **0-address:** `ADD` ⇒ pop two, push sum — *only* the opcode; every operand implicit from the stack top; stack machines, JVM.

  Teaching note: The word **destructive** (2-address) needs an explicit example, not just a definition — students accept "`ADD R1,R2` loses `R1`'s old value" only when they see it force an extra `MOV` into the program (below). Tie it forward: this is *why* the 2-address program is longer than the 3-address one.

Now the running thread, compiled in every format. Write each tally on the board and let the class count with you.

Example 4.4.1 (3-address: `X = (A+B)×(C+D)`). *`ADD R1, A, B` ⇒ $R1 \leftarrow A + B$; `ADD R2, C, D` ⇒ $R2 \leftarrow C + D$; `MUL X, R1, R2` ⇒ $X \leftarrow R1 \times R2$.*
***3 instructions**, each carrying three addresses — wide words, short program, no extra data movement.*

Example 4.4.2 (2-address: the same expression). *`MOV R1, A; ADD R1, B; MOV R2, C; ADD R2, D; MUL X, R1` (R2 is the implicit second operand of `MUL`).*
***5 instructions** vs. 3 — the 2-address format is cheaper per instruction but needs extra copy instructions because `ADD` is destructive.*

Example 4.4.3 (1-address: the same expression). *`LOAD A; ADD B; STORE T; LOAD C; ADD D; MUL T; STORE X` — with a temporary `T` in memory, because the accumulator cannot hold two partial results at once.*
7 instructions.

Example 4.4.4 (0-address: the same expression). *`PUSH A; PUSH B; ADD; PUSH C; PUSH D; ADD; MUL; POP X` — shortest instructions, but the most of them.*
8 instructions.

A question worth asking live, after the 3-address tally: "what is the one instruction the 3-address program never needs that the 2-address program does?" (The `MOV`/`LOAD` copies — 3-address has no destructive overwrite to repair.) Then put all four counts on the board at once (Table 2).

Format	**Example**	**Program size for $X = (A+B) \times (C+D)$**	**Operand access**	**Typical use**
3-address	`ADD R1,R2,R3`	3 instr., wide words	all explicit	register ISAs (RISC)
2-address	`ADD R1,R2`	5 instr. + copies	one destroyed	older CISC
1-address	`ADD A`	7 instr., needs temps	accumulator	simple CPUs
0-address	`ADD`	8 instr., compact	stack top	stack machines, JVM

Table 2: The four formats side by side — cf. Mano, Sec. 8-4, p.255.

Teaching note: Resist the urge to call any format "best." The class will want a winner. The correct takeaway is the monotone trade-off: fewer addresses ⇒ more instructions but compact words; more addresses ⇒ fewer instructions but wide words. RISC uses 3-address; the JVM uses 0-address; both are correct for their design goals.

4.5 Instruction Cycle

Set up the section with the Socratic question: `ADD R1, A` sits in memory; for it to take effect the CPU must bring it in, understand it, collect `[R1]` and `[A]`, add, and store back. What fixed sequence of basic steps does the machine repeat for *every* instruction? The answer is the **instruction cycle** — a skeleton the control unit runs once per instruction, identical for every instruction except that the **execute** phase varies by opcode.

Definition (Instruction Cycle). *The fixed sequence of phases — fetch, decode, execute — the control unit repeats once per instruction; only the execute phase varies with the opcode.*

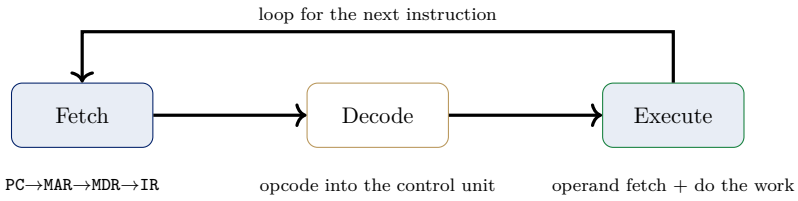

Figure 4.4: The instruction cycle — fetch, decode, execute — as a loop (Hamacher, *Computer Organization*, Ch. 7; standard treatment).

How to present it: draw the three boxes left to right first, then the return arrow across the top last — and narrate the top arrow as the thing that makes it a *cycle*: "after execute, we go back and fetch the next instruction." The three sub-captions under the boxes (`PC→MAR→MDR→IR`; opcode into control unit; operand fetch + do the work) should each be revealed as you explain that phase.

> **Teaching note:** Students often think decode and execute are the "interesting" phases and fetch is trivial bookkeeping. The teaching move is the reverse emphasis: fetch is the *identical* prologue every instruction runs, and it is where `PC`, `MAR`, `MDR`, `IR` all earn their keep at once. Spend the time on fetch's five transfers; execute's variety is then a natural surprise rather than a laundry list.

Spell out fetch's five transfers (the sub-caption under the Fetch box, expanded): `MAR` ← `[PC]`; initiate a memory read at `MAR`; `MDR` ← *memory data*; `IR` ← `[MDR]`; `PC` ← `[PC]` +1. Each register exists for *one* reason: `PC` holds the next address, `MAR` pins it during the access, `MDR` buffers the incoming word, `IR` holds the current instruction while it is decoded.

A question worth asking live, after the five transfers: "why does the instruction go `PC` → `MAR` → `MDR` → `IR` instead of straight `PC` → `IR`?" (Because `PC` and `IR` are not connected by a direct path in a bus machine — the memory interface needs the address pinned in `MAR` for the whole access and the word buffered in `MDR` before it is stable enough to load into `IR`.)

Then decode and execute. Decode: the opcode bits of `IR` tell the control unit *which* operation this is — it then selects the right sequence of control signals (exactly what Weeks 6–7 build). Execute: the operand-dependent part — if a memory operand is needed, its address is resolved (addressing modes, Week 5) and the operand fetched into a register; the ALU performs the operation; the result is written to the destination named in the instruction. The *number* of execute steps varies by instruction — fetch is fixed, execute is not.

Example 4.5.1 (One full cycle for `ADD R1, A`).

Phase	*Register activity*	*Memory / ALU*	*State*
Fetch	*MAR←[PC]; MDR←instr.; IR←[MDR]; PC←[PC]+1*	*read instruction word*	*instruction in IR*
Decode	*IR opcode → control unit*	—	*operation identified*
Execute	*operand [A] fetched; AC←[AC]+[A]*	*read operand; ALU adds*	*result in AC*

The same three phases run for every instruction — only the execute row changes.

Example 4.5.2 (A Second Instance: `STORE X` on the same machine)**.** *Fetch (identical five transfers, PC incremented); decode (opcode = STORE); execute (the operand [X]'s* address *is resolved, the value to store is already in AC, and the memory write is issued — note the execute row here writes* out *rather than reading in, so its register activity and memory column differ completely from ADD). Use this contrast to make the point that only the execute row varies.*

Now the four types of operands (Hamacher, Ch. 7; standard treatment): **addresses** (a location in memory — used by `LOAD`, `STORE`, `BRANCH`); **numbers** (integer or floating-point values the ALU computes on); **characters** (text — usually ASCII/Unicode codes in a byte or halfword); **logical data** (bit patterns — each bit its own value; `AND`, `OR`, `XOR`, shifts work bitwise). Why this matters: the *type* determines how the ALU wires the bits — a branch needs the address, an integer add needs the value, a shift needs the bit mask — and the opcode says which interpretation applies (P&H makes the same operand taxonomy in Sec. 2.3, p.67).

> **Teaching note:** The four operand types are the section's "so what." Without them, the cycle looks like pointless plumbing; with them, the fetch–decode–execute skeleton becomes *the* engine that interprets whichever kind of data the opcode names. Connect each type to the ALU operation it feeds (branch → address, add → value, shift → mask) as you list it.

Close the section by tying the week together on the board: for `ADD R1, A` on an accumulator machine, `PC` points at the next instruction; `IR` holds the current one after fetch; the first operand is `AC` (the implicit operand of the 1-address format); the second is memory location `A`, fetched during execute. Every piece of that answer came from the register organization, the formats, and the cycle — one connected machine.

4.6 Synthesis and Summary

Storage spaces and formats are two sides of one choice: **registers** (fast, explicit, random access) buy the wide 3-address format; **stack** (compact, restricted, implicit) buys the 0-address format; the instruction cycle is the glue that steps whichever format the ISA chose through the same fetch/decode/execute skeleton. The running thread closes: $X = (A + B) \times (C + D)$ ran in 3, 5, 7, or 8 instructions depending on the format, and each of those instructions then lives through one full fetch–decode–execute cycle.

Set up next week's hook explicitly: we wrote "`ADD R1, A`" and said "operand `[A]` is fetched" — but never how the field `A` turns into the actual location of the operand. That is **addressing modes**, Week 5, together with the datapath (buses) that actually moves the values.

Summary — quick reference: *Register organization:* registers are the one-cycle scratchpad; transfers happen over a common bus selected by `SELA`/`SELB`/`SELD`. *Stack organization:* `SP` addresses

a LIFO top; PUSH/POP with ±1 on `SP`; register stacks fast, memory stacks large. *Instruction formats:* 3-, 2-, 1-, 0-address — the address count trades instruction width against program size (3 vs. 5 vs. 7 vs. 8 instructions for one expression). *Instruction cycle:* fetch (`PC`→`MAR`→`MDR`→`IR`, `PC`+1), decode, execute. *Operand types:* addresses, numbers, characters, logical data — the opcode picks the interpretation.